

Challenge Overview

The challenge provides a binary implementing a 21×21 maze representing the "Minotaur's Bed & Breakfast". The objective is to navigate from the starting point **LOBBY (S)** to the destination **PENTHOUSE (E)** using classic **WASD** controls over a remote netcat service (**nc 103.213.38.195 8056**).

Vulnerability / Mechanism Analysis

Inspection of the binary strings reveals fragments of a layout stored sequentially in memory using the characters **a** (representing walkable floor paths) and **l** (representing walls):

Plaintext

```
aaaaaaaaaaaaaaaaaaaaa
aaalalalaaalalaaaaa
alalalaaaaalallaaala
alaaaaalaaaaaalalaaa
alalaaaaaaaaalaaala
aaaaaaaaaaaaaaaaaaaaa
```

The program reads all movement inputs simultaneously via **fgets** into a buffer. When providing a single directional input step-by-step, the binary terminates the session immediately upon displaying the outcome of that isolated turn. This prevents conventional interactive online Depth-First Search (DFS) script techniques from backtracking dynamically within the same network session.

However, because the server remains active when evaluated against long pooled string arrays, we can exploit the application's verbosity. If a sequence hits a wall, the remote instance precisely returns the index at which the collision happened (e.g., **Guest ran into a wall at turn X**).

Exploitation Strategy

To efficiently determine the path without manually guessing the entire 21×21 layout, an automated Python execution script was designed to perform **Dynamic Blind Maze Discovery**:

1. **State Persistence**: Maintain a local **visited_positions** coordinate cache tracking **(X, Y)** positions relative to an arbitrary origin.
2. **Pre-Validation Branching**: Iterate through prospective moves (**w, s, a, d**). For each move, look up the target relative coordinates to confirm it hasn't been traveled yet (preventing endless back-and-forth loops inside corridors).

3. **Index Fault Analysis:** Transmit the tentative path accumulation directly to the remote server.
 - If the response contains a wall crash flag precisely matching the current string length (`turn LEN(path)`), mark that coordinate block as a wall locally and try a different direction.
 - If the index passes without a wall error, accept the path step, append the coordinate to the visited set, and continue iterating forward.
4. **Backtracking:** If a deadlock occurs (all four directions lead to walls or previously explored tiles), drop the last character from the string array and fallback to evaluate alternative paths.

Solution Script

Python

```
from pwn import *
```

```
context.log_level = 'error'
```

```
host, port = '103.213.38.195', 8056
```

```
# Coordinate deltas
```

```
MOVES = {
```

```
    'w': (-1, 0),
```

```
    's': (1, 0),
```

```
    'a': (0, -1),
```

```
    'd': (0, 1)
```

```
}
```

```
current_path = "s"
```

```
visited_positions = {(0, 0), (1, 0)}
```

```
while True:
```

```
    found_move = False
```

```
    r, c = 0, 0
```

```
    for step in current_path:
```

```
        dr, dc = MOVES[step]
```

```
        r += dr
```

```
        c += dc
```

```
    for move in ['s', 'd', 'a', 'w']:
```

```
        dr, dc = MOVES[move]
```

```
        next_pos = (r + dr, c + dc)
```

```
    if next_pos in visited_positions:
```

```
        continue
```

```

test_sequence = current_path + move

try:
    io = remote(host, port)
    io.recvuntil(b"(WASD):")
    io.sendline(test_sequence.encode())
    response = io.recvall(timeout=1).decode()
    io.close()
except Exception:
    continue

if "CYBERNAUTS{" in response or "secret" in response:
    print(f"\n[+] Winning Path found: {test_sequence}")
    print(response)
    exit(0)

wall_signature = f"turn {len(test_sequence)}"

if wall_signature not in response and response != "":
    current_path = test_sequence
    visited_positions.add(next_pos)
    found_move = True
    break

if not found_move:
    if len(current_path) > 0:
        current_path = current_path[:-1]
    else:
        break

```

Flag

Providing the resolved sequence

(ssddssssssssaassdddssaassaassdddwwddddddssddwwwdddssss) bypasses the maze logic completely and triggers the flag dump:

CYBERNAUTS{m1n0t4ur_bnb_th3s_14byr1nth_f0und_th3_p3nth0us3}

```
[+] Advanced to tile 51: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwww
[+] Advanced to tile 52: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwww
[+] Advanced to tile 53: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwwwd
[+] Advanced to tile 54: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwwwdd
[+] Advanced to tile 55: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwwwddd
[+] Advanced to tile 56: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwwwddds
[+] Advanced to tile 57: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwwwddds
[+] Advanced to tile 58: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwwwddds
[+] Advanced to tile 59: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwwwddds
```

```
[+] Flag Found! Path: ssddsssssssaassdddddssaasssaassdddddwwddddddssddwwwddds
You navigated the labyrinthine 8n8! The Minotaur reveals the secret:
CYBERNAUTS{mIn0t4ur_bnb_th3s_l4byr1nth_f0und_th3_p3nth0us3}
```

```
(base) └─(kali@kali)-[~/Desktop/NSU]
└─$ █
```